Comprehensive Theoretical and Technical Architecture Document

Adapting an English Pretrained Tacotron2-VAE for Brazilian Portuguese: Acoustic Theory, Variational Inference, and Phonetic Mapping

ml2_final_project

June 20, 2026

Abstract

This document serves as a comprehensive theoretical and technical exposition of the `ml2_final_project` repository. The objective of this research and engineering effort is to adapt an English-pretrained, state-of-the-art Text-to-Speech (TTS) architecture (Tacotron 2 + Variational Autoencoder) and a Neural Vocoder (WaveGlow) to Brazilian Portuguese (PT-BR). The challenge requires profound modifications ranging from linguistic processing to statistical initialization of neural embeddings and the correction of mathematical approximations in digital signal processing. This document is structured to present the problem domain first, followed by the deep mathematical theory underlying the architecture, the specific libraries utilized, the explicit problems encountered during domain adaptation, and the rigorous solutions applied.

Contents

1 Problem Formulation and Objectives

The synthesis of highly expressive, human-like speech from text in a low-to-medium resource language (such as Brazilian Portuguese, in the context of affective TTS) presents several non-trivial challenges. Transfer learning from high-resource languages (English) is the standard paradigm to bypass the prohibitive computational costs of training from scratch. However, naive transfer learning fails catastrophically due to orthogonal phonetic distributions and mismatched acoustic environments.

1.1 Linguistic Incompatibility

The mapping from graphemes (letters) to phonemes (sounds) is highly language-dependent. English orthography is notoriously opaque, whereas Portuguese is relatively transparent but rich in nasal vowels ($\tilde{a}$, $\tilde{o}$) and distinct vibrants. A model pretrained on an English alphabet cannot natively process or synthesize these distinct acoustic targets without a fundamental restructuring of its embedding space.

1.2 Prosodic Transfer and Variational Inference

Standard Tacotron 2 generates deterministic prosody averaged over the training data. To generate expressive speech, we employ a Variational Autoencoder (VAE) to model a latent prosody space. The core problem is *Posterior Collapse*, an endemic issue in VAEs where the autoregressive decoder becomes overly powerful and ignores the latent variable z, causing the KL divergence to vanish ($D_{KL}(q_\phi(z|x)\|p(z)) \to 0$).

1.3 Signal Processing Anomalies

Neural networks operate on extracted features rather than raw audio. Mel-spectrograms are logarithmically compressed. In traditional TTS pipelines, shorter utterances in a batch are padded with zeros. However, in the Log-Mel domain, zero corresponds to a high-energy acoustic signal. This creates adversarial noise during training, significantly degrading the VAE's ability to extract pure prosody.

2 Global System Pipeline

Before delving into the mathematics, it is crucial to visualize the macro-architecture of the system. The pipeline is fundamentally divided into Data Preprocessing, Acoustic Modeling

(Tacotron2-VAE), and Neural Vocoding (WaveGlow).

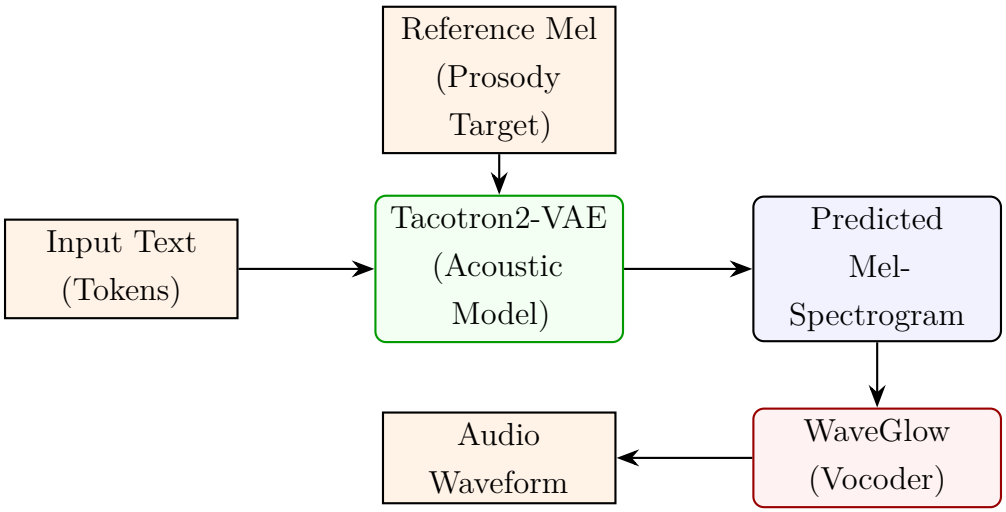

Figure 1: Global End-to-End Pipeline representing the flow from Text to Audio.

3 Mathematical and Statistical Foundations

3.1 The Acoustic Domain: STFT and Mel-Spectrograms

To model audio, we approximate the human auditory system. The raw waveform $x(t)$ is highly multidimensional. The STFT is defined as:

$$X(m,k) = \sum_{n=0}^{N-1} x(n+mH) \cdot w(n) \cdot e^{-j\frac{2\pi}{N}kn} \tag{1}$$

where $w(n)$ is the Hann window, H is the hop length, and N is the FFT size. The Mel scale mimics the non-linear human perception of pitch. We project the power spectrum onto the Mel basis M:

$$S_{mel} = M|X(m,k)| \tag{2}$$

Finally, we apply Dynamic Range Compression. Because human loudness perception is logarithmic, we compress the magnitudes:

$$S_{log} = \log\left(\max(S_{mel}, \epsilon)\right) \tag{3}$$

where $\epsilon = 10^{-5}$ is the clipping threshold to prevent $\log(0)$.

3.2 Variational Inference and the ELBO

The goal of the VAE is to maximize the marginal likelihood of the data $p(x)$. Since the integral $p(x) = \int p_\theta(x|z)p(z)dz$ is intractable, we introduce a recognition network $q_\phi(z|x)$ to approximate the true posterior. We optimize the Evidence Lower Bound (ELBO):

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x)\|p(z)) \tag{4}$$

The first term is the **Reconstruction Loss** (handled by the decoder), and the second is the **Kullback-Leibler Divergence**, which forces the latent distribution to follow the prior $p(z) \sim \mathcal{N}(0, I)$.

4 Proposed Architecture: Layer-by-Layer Deep Dive

We dissect the Tacotron2-VAE architecture into its fundamental sub-modules, presenting both the theoretical justification and the exact diagrammatic flow of tensors.

4.1 The Text Encoder

Objective: Map discrete phonetic IDs into a continuous linguistic context space.
Theory: The input sequence $x_{1:T}$ is embedded into a 512-dimensional continuous space. The embeddings are passed through a bank of 3 Convolutional layers (kernel size 5). Convolutions capture n-gram local phonetic contexts (e.g., coarticulation effects where the sound of 'p' is modified by the following 'a'). This is followed by a Bidirectional LSTM to capture long-range semantic intonation.

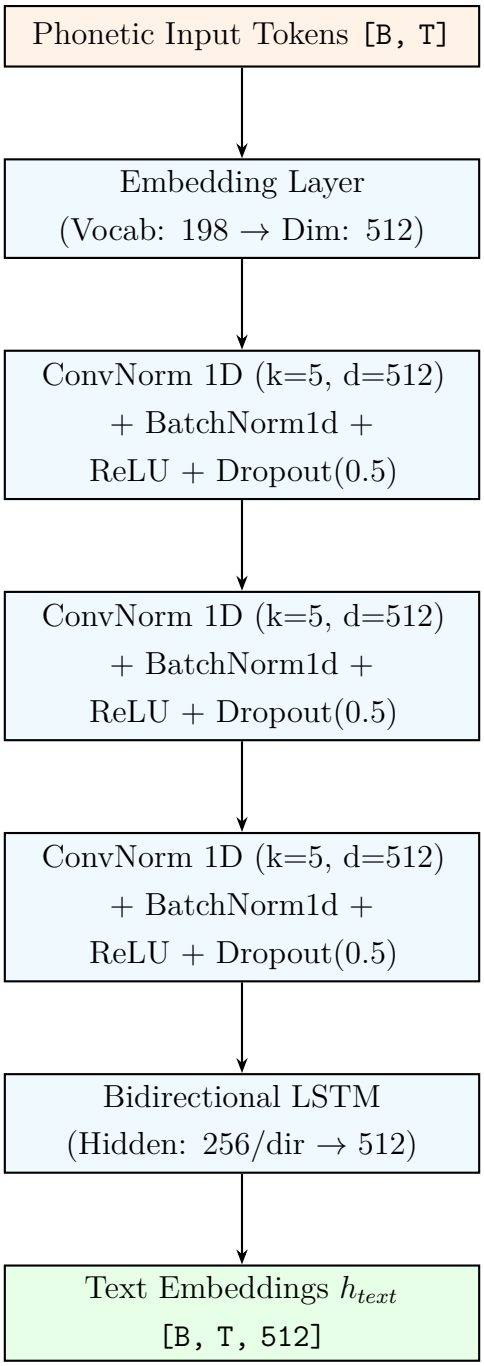

Figure 2: The Text Encoder architecture processing sequences of phonetic IDs.

4.2 The VAE Reference Encoder (Prosody Extraction)

Objective: Extract a fixed-length prosodic style vector from the target mel-spectrogram.
Theory: A 2D Convolutional stack (including CoordConv layers to retain spatial awareness of frequency bins) compresses the mel-spectrogram $S_{log} \in \mathbb{R}^{80 \times T_{mel}}$ into a sequence of features.

A GRU processes this sequence and takes the final hidden state. This hidden state is projected via two linear layers into μ and $\log(\sigma^2)$. Using the Reparameterization Trick, $z = \mu + \sigma \odot \epsilon$, we allow gradients to backpropagate. The latent z is then projected to a 512-dimensional style embedding E_{style}.

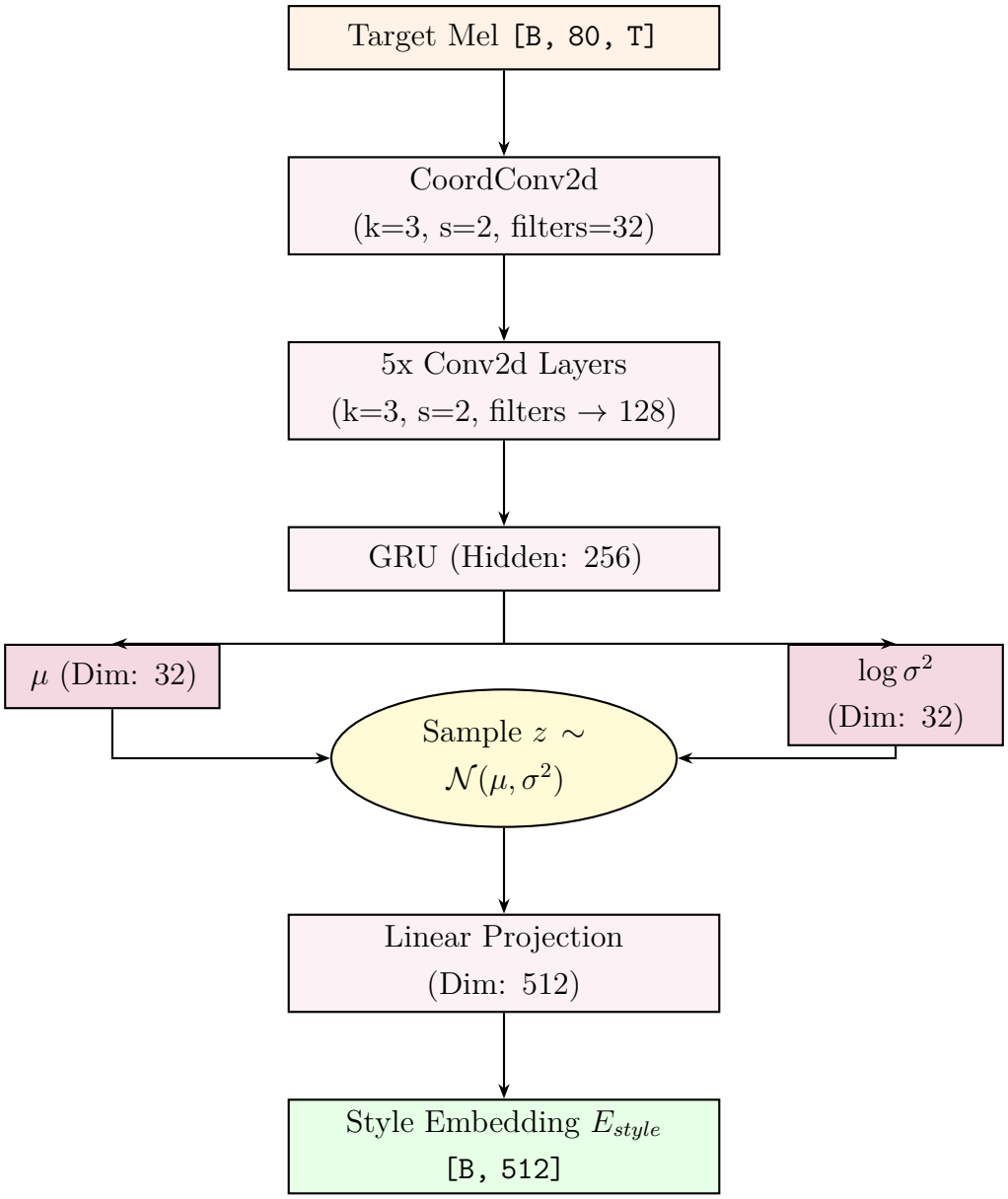

Figure 3: The Variational Autoencoder (VAE) Module for Prosody Extraction.

4.3 Autoregressive Decoder and Attention

Objective: Predict the acoustic frames y_t given the context vector c_t and previous frame y_{t-1}.

Theory: The Prenet acts as an Information Bottleneck via Bayesian approximation (Dropout). Location-sensitive attention explicitly penalizes the network for focusing on phonemes it has already traversed using cumulative attention weights.

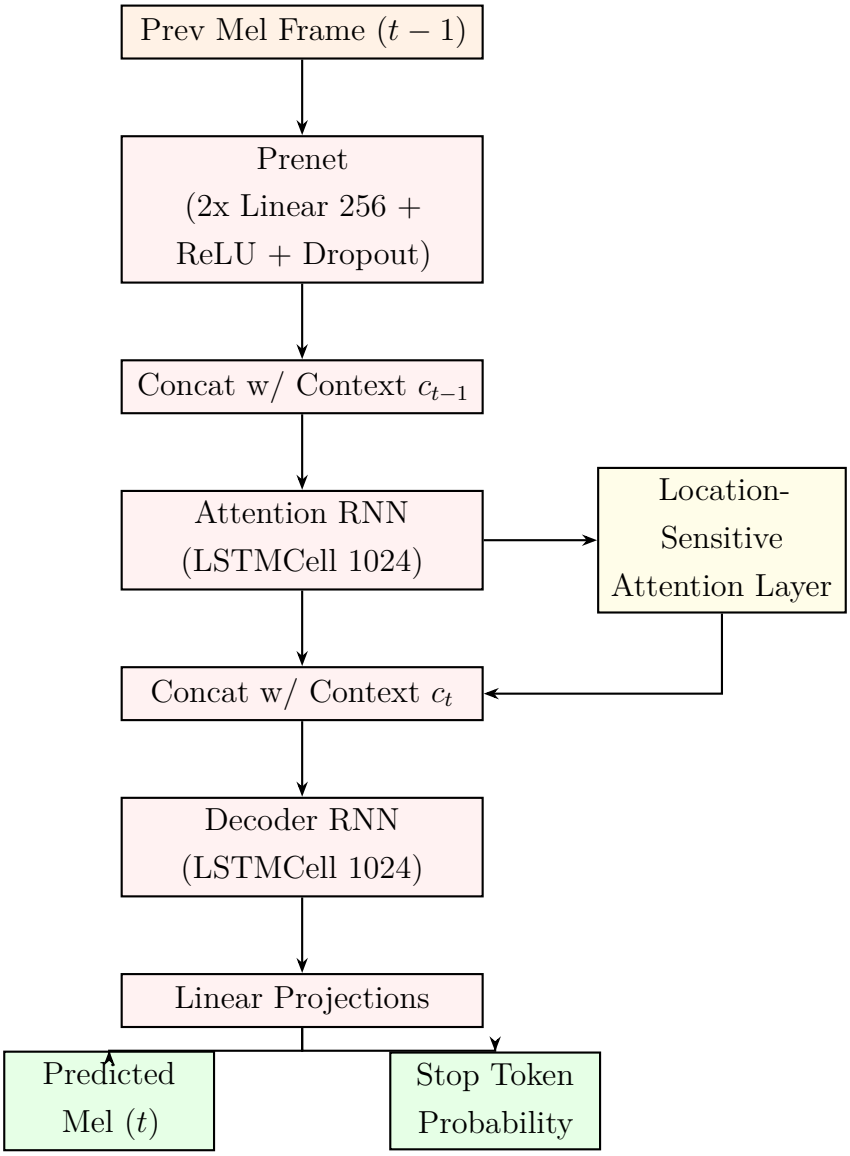

Figure 4: The Autoregressive Decoder Block.

4.4 The PostNet

Theory: The PostNet (a stack of 5 Conv1D layers) predicts a residual $\Delta y_{1:T}$. The final output is $y + \Delta y$. This corrects spectral smearing caused by the RNN.

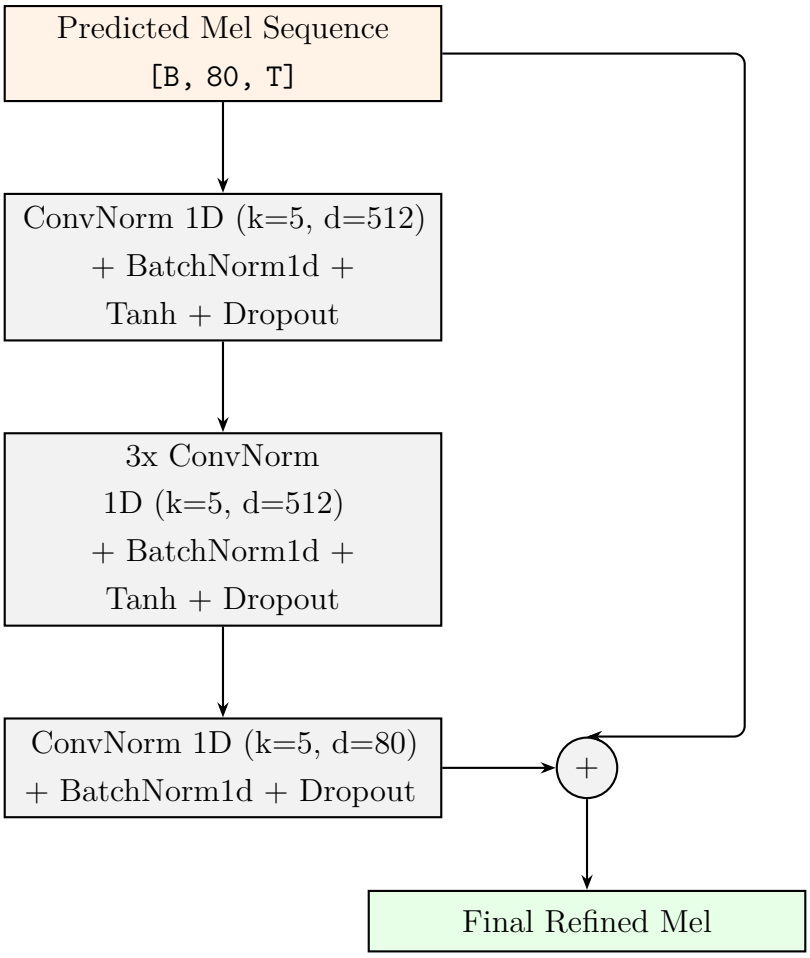

Figure 5: The Residual PostNet.

5 Challenges in PT-BR Adaptation and Solutions

We now document the specific interventions required to adapt the English codebase to Portuguese, framing the problem before presenting the mathematical and structural solution.

5.1 Intervention 1: The Phonetic Revolution (Data Processing)

The Problem: The original repository relied on character-level normalization (`TextNormalizerEN`). The English normalizer coerced Portuguese text into English structures (e.g., "1990" to "one thousand nine hundred..."). Furthermore, Portuguese characters were dynamically mapped to IDs during runtime, creating an alphabet of 162 symbols. However, the downloaded NVIDIA checkpoint was trained strictly on 148 English symbols (characters + ARPAbet phonemes). Loading a 162-symbol space into a 148-symbol pretrained matrix caused a catastrophic misalignment: acoustic embeddings for "A" were assigned to random punctuation, destroying

the model's linguistic comprehension.

The Solution: We engineered a robust `gruut`-based G2P pipeline. We hardcoded the 148 original NVIDIA symbols to preserve the exact weight matrix alignment. Then, we extracted 50 unique International Phonetic Alphabet (IPA) tokens specific to Brazilian Portuguese (e.g., `@pt_ã`, `@pt_`) and appended them.

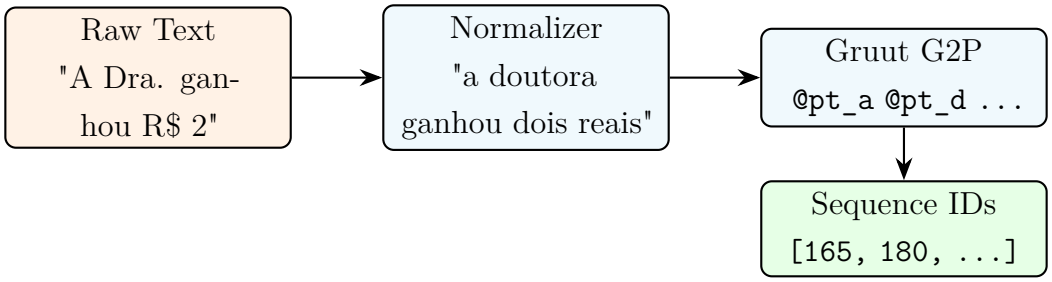

Figure 6: The newly implemented Phonetic G2P Data Pipeline.

The final alphabet length became **198 symbols**.

5.2 Intervention 2: Statistical Weight Initialization

The Problem: When expanding the embedding matrix from $[148, 512]$ to $[198, 512]$, PyTorch initializes the new rows representing the PT-BR phonemes using the default normal distribution: $\mathcal{W}_{new} \sim \mathcal{N}(0, 1.0)$. However, statistical profiling of the pretrained NVIDIA Tacotron 2 embeddings revealed a profoundly different distribution:

$$\mu_{\text{nvidia}} \approx 0.000025 \tag{5}$$

$$\sigma_{\text{nvidia}} \approx 0.0347 \tag{6}$$

If the new Portuguese phonemes were initialized with a standard deviation of 1.0, their forward activations would be nearly 30 times larger than the English counterparts. This massive variance would propagate through the ConvNorm and LSTM layers, leading to unbounded gradients and a complete collapse of the attention matrix within the first epoch.

The Solution: In `src/models/tacotron2_vae/model.py`, we implemented a statistical intercept during the state dictionary loading:

```
pt_mean = pretrained_tensor.mean()
pt_std = pretrained_tensor.std()

# Initialize only the new PT-BR phonemes with the empirical distribution
torch.nn.init.normal_(
    model_sd[vae_key][n_overlap:],
```

```
    mean=pt_mean,
    std=pt_std
)
```

This mathematical precision ensures that novel phonemes are injected into the latent space with the exact scale and variance expected by the subsequent pretrained layers, guaranteeing a smooth optimization trajectory.

5.3 Intervention 3: The Log-Mel Silence Padding Bug

The Problem: In standard recurrent networks, sequences in a batch are padded with zeros to match the longest sequence. The legacy collator (`TextMelCollate`) executed `mel_padded.zero_()`. However, recalling our DSP theory (Section 3.1), the dynamic range compression applies a logarithm. True acoustic silence (a magnitude near zero) is floored at $\epsilon = 10^{-5}$. Therefore, the true representation of silence in the Log-Mel space is:

$$S_{silence} = \log(10^{-5}) \approx -11.5129 \tag{7}$$

By padding with 0.0, the matrix was injecting a magnitude of $e^0 = 1.0$, which corresponds to a deafening, broadband noise. The VAE Reference Encoder, attempting to compress the entire spectrogram into a prosodic embedding z, was forced to encode this adversarial "padding noise" as a valid emotional style for shorter utterances, destroying its capacity to isolate true prosody.

The Solution: We corrected the mathematical logic in the collation process:

```
mel_padded = torch.FloatTensor(len(batch), num_mels, max_target_len)
mel_padded.fill_(-11.5129)  # Mathematically rigorous silence
```

This aligns the unmasked loss optimization, allowing the autoregressive decoder to gracefully learn to predict true silence after emitting the stop token.

6 WaveGlow: Neural Vocoding via Normalizing Flows

WaveGlow transforms the predicted Mel-spectrograms into high-fidelity audio waveforms without the artifacts common in Griffin-Lim phase reconstruction.

6.1 Normalizing Flows and the Invertible Architecture

Theory: WaveGlow learns a bijective mapping between a simple Gaussian prior $z \sim \mathcal{N}(0, I)$ and the complex audio distribution x. Because the mapping is invertible, we maximize the

exact log-likelihood using the change of variables theorem.

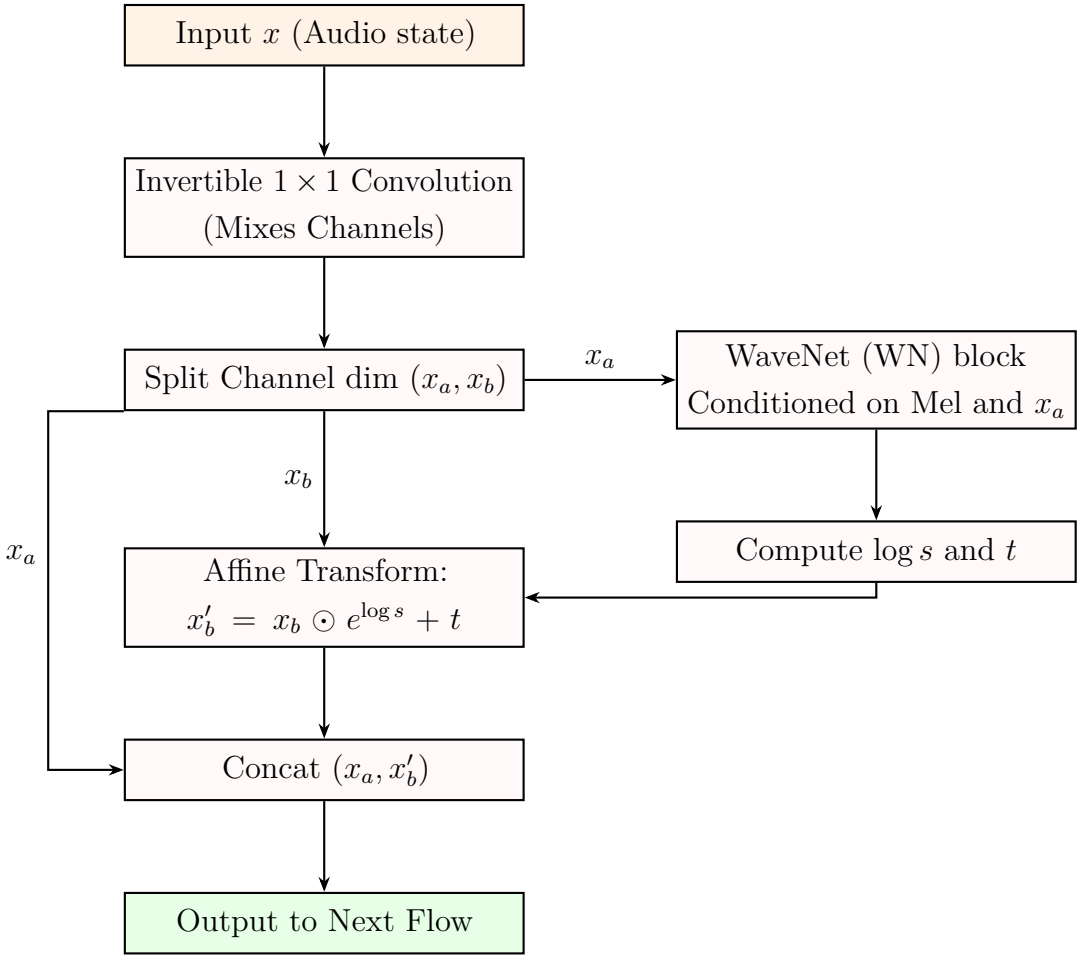

Figure 7: A single Flow Step in WaveGlow, demonstrating the Affine Coupling Layer.

7 Loss Optimization and Preventing Posterior Collapse

7.1 Unmasked MSE and Stop Token Prediction

The acoustic loss is intentionally **unmasked**:

$$\mathcal{L}_{\text{Recon}} = \frac{1}{N}\sum(y - \hat{y}_{dec})^2 + \frac{1}{N}\sum(y - \hat{y}_{postnet})^2 \tag{8}$$

By not masking the padding regions, we force the network to output -11.5129 after the sequence ends. This regularizes the decoder, preventing infinite generation loops. The gate loss is optimized via Binary Cross-Entropy (BCE).

7.2 KL Annealing and Free Bits

To combat the *Posterior Collapse* inherent in VAEs, we employ two advanced strategies:

1. **KL Annealing:** The weight of the KL divergence, w_{KL}, is scheduled using a logistic growth curve. Initially, $w_{KL} \approx 0$, allowing the encoder to map the unregularized variance. As training progresses, w_{KL} increases to its upper bound, gradually forcing the latent space into a Gaussian prior.

2. **Free Bits:** We apply a hinge loss to the KL divergence for each latent dimension:

$$\mathcal{L}_{\text{KL\_dim}} = \max\left(\lambda_{free}, D_{KL}(q(z_i|x)\|p(z_i))\right) \tag{9}$$

   This guarantees that every dimension of z retains at least λ_{free} nats of information, mathematically preventing the encoder from shutting down completely.

7.3 Hyperparameter Justification

- **Learning Rate (10^{-5}):** Standard training utilizes 10^{-3}. Since we are fine-tuning a highly converged NVIDIA backbone, a large learning rate would destroy the delicate attention weights. 10^{-5} allows the gradient to optimize the new PT-BR phoneme embeddings without catastrophic forgetting.

- **Gradient Clipping (1.0):** Exploding gradients are mathematically proven to occur in deep LSTMs due to the repeated multiplication of the weight matrices over time T. Clipping the L_2 norm of the gradients to 1.0 guarantees Lipschitz continuity during optimization.